

Supervised Learning

Feature space expansions

Mahdi Shafiee Kamalabad and Maarten Cruyff

1. Linearity
2. Polynomials
3. Splines
4. Regression trees

Feature space expansions

- In supervised learning, we model the relationship between inputs X and an outcome Y .
- Feature space expansion means transforming the original input features into a richer set of features (or a higher-dimensional space).
- This allows simple models (e.g. linear models) to capture non-linear and complex relationships.
- Different methods expand the feature space in different ways:
 - explicitly (creating new features, e.g. polynomials, splines),
 - or implicitly (using kernel methods, e.g. kernel regression).

Linearity

Simple Linear model

The simple linear regression model

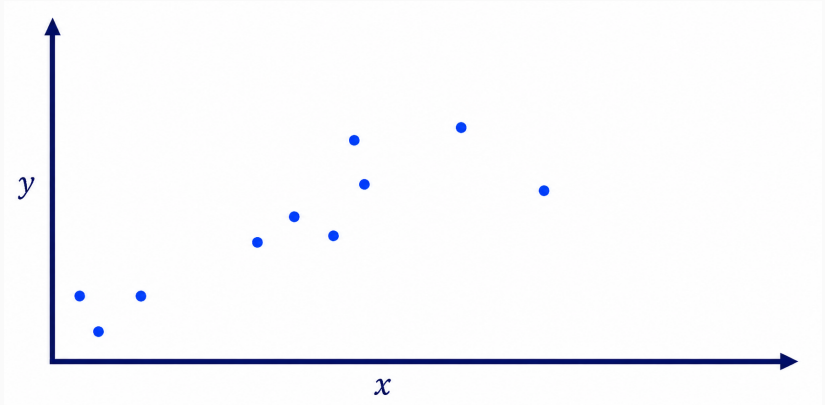
$$y = \beta_0 + \beta x + \varepsilon, \quad \varepsilon \sim N(0, \sigma^2)$$

- The mean relationship between x and y is linear (a straight line).
- Predictions lie on this fitted line:

$$\hat{y} = \hat{\beta}_0 + \hat{\beta}_1 x$$

- Errors, ϵ , (approximated by residuals, e) are assumed:
 - Normally distributed (around 0)
 - Constant variance (homoscedasticity)

Observed data before model fitting



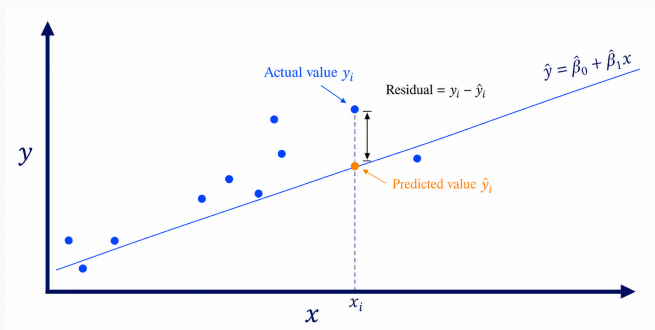
The fitted linear model $\hat{y} = \hat{\beta}_0 + \hat{\beta}_1 x$

$\hat{\beta}_0, \hat{\beta}_1$ are their estimates from the data,

$\hat{y}$ is the predicted (fitted) value.

y is the actual value.

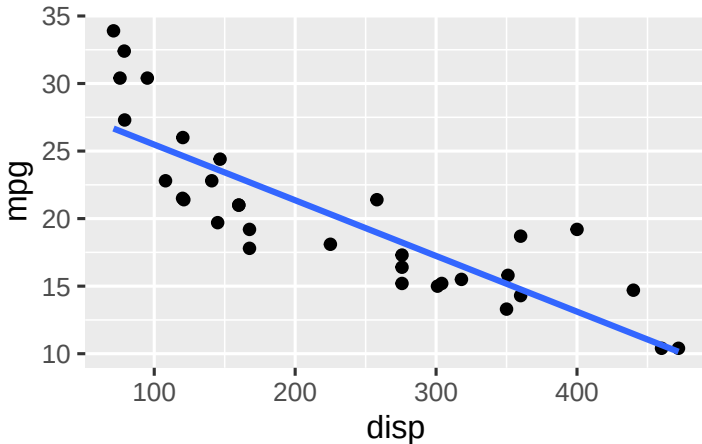
$$\text{Residual} = \text{Actual} - \text{predicted} = y - \hat{y}$$



What if x,y relationship is non-linear

Example: predicting fuel efficiency from engine size (data = mtcars)

- mpg: miles per gallon (fuel efficiency)
- disp: engine displacement (engine size)
- Linear regression fit (blue line) assumes a straight-line relationship

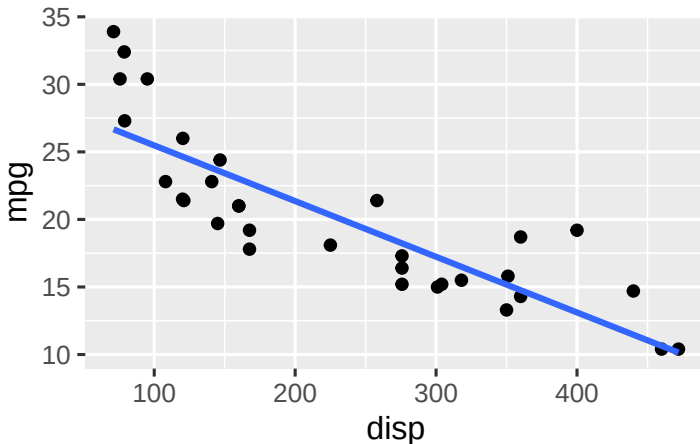


**Is a simple linear model always
the Best Choice? What can go
wrong?**

What if x,y relationship is non-linear

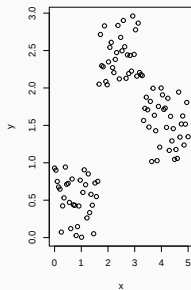
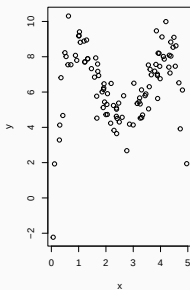
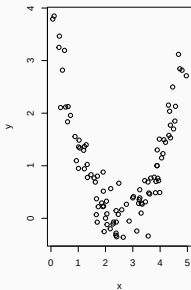
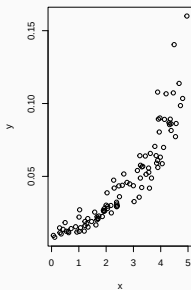
What goes wrong with the simple linear model?

- Underestimates fuel efficiency at very small and very large engines (at the extremes of `disp`)
- The scatter plot suggests the true relationship may be non-linear



Non-linearity

- Not all relationships are well described by straight lines.
- Non-linear patterns can take many shapes.
- Different shapes require different modeling approaches.



We need models that can capture nonlinear patterns.

Accommodating non-linearity

A linear model can handle non-linear relationships by changing how x enters the model.

- **polynomials**

- Add powers of x
- Produces smooth curved regression lines

$$y = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3 + \dots$$

- **splines**

- Fit different polynomials in different regions of x
- Pieces are joined smoothly
- More flexible than a single polynomial

- **A different approach is a regression tree**

- Split the input space into non-overlapping regions
- Predict using the mean response within each region
- No smoothness assumption

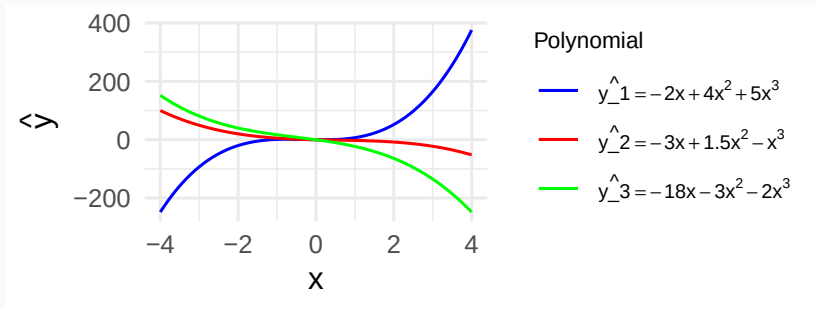
Polynomials

Basis expansion

We make the model more flexible by adding powers of x .

- For example the cubic polynomial

$$\hat{y} = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3$$



- Still a linear model in the coefficients
- Can produce many different curved shapes, depending on the coefficients.

Making polynomials in R: `I()` inside `lm()`

The straightforward approach

- Use `I()` inside `lm()` to add polynomial terms.
- `I()` tells R to treat x^2 , x^3 , etc. as mathematical expressions, not as formula operators.
- R internally creates the polynomial basis using a `model matrix`.
- `model.matrix()` shows the expanded feature space used by the model.

```
(M <- model.matrix(~ I(x^1) + I(x^2) + I(x^3), data.frame(x = 1:4)))
```

```
(Intercept) I(x^1) I(x^2) I(x^3)
1           1      1      1      1
2           1      2      4      8
3           1      3      9     27
4           1      4     16     64
attr(,"assign")
[1] 0 1 2 3
```


A toy example: advertising and sales

Suppose a small business records weekly advertising spending and sales.

- `advertising`: advertising spend level
- `sales`: number of sales
- Note: We still do not know which model is the best model here. We will discuss about this later.

A toy example: advertising and sales

Data: advertising spend and sales

```
# A tibble: 6 x 2
  advertising sales
      <int> <dbl>
1         1     2
2         2     5
3         3    10
4         4    17
5         5    26
6         6    37
```

- statistical model:

$$y = \beta_0 + \beta_1 x + \beta_2 x^2 + \varepsilon, \quad \varepsilon \sim N(0, \sigma^2)$$

- fitted model:

$$\hat{y} = \hat{\beta}_0 + \hat{\beta}_1 x + \hat{\beta}_2 x^2$$

A toy example: advertising and sales

- statistical model

$$\text{sales} = \beta_0 + \beta_1 \text{advertising} + \beta_2 \text{advertising}^2 + \varepsilon$$

- fitted model:

$$\widehat{\text{sales}} = \hat{\beta}_0 + \hat{\beta}_1 \text{advertising} + \hat{\beta}_2 \text{advertising}^2$$

R code

```
fit <- lm(sales ~ advertising + I(advertising^2), data = myData)

summary(fit)

Call:
lm(formula = sales ~ advertising + I(advertising^2), data = myData)

Residuals:
    1         2         3         4         5         6 
9.618e-17  1.667e-16 -8.648e-16  6.325e-16  1.818e-16 -2.123e-16 

Coefficients:
                Estimate Std. Error  t value Pr(>|t|)
(Intercept)    1.000e+00  1.161e-15  8.615e+14  <2e-16 ***
advertising   -1.554e-15  7.594e-16 -2.047e+00   0.133
I(advertising^2) 1.000e+00  1.062e-16  9.417e+15  <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 6.489e-16 on 3 degrees of freedom
Multiple R-squared:      1, Adjusted R-squared:      1
F-statistic: 1.063e+33 on 2 and 3 DF,  p-value: < 2.2e-16
```

Interpreting the results

The fitted model is:

$$\widehat{\text{sales}} = 1 + \text{advertising}^2$$

This means the relationship is **curved**, not linear.

**What can go wrong with
polynomial models?**

A Potential problem: Multicollinearity

- Polynomial terms such as x, x^2, x^3 are often highly correlated.
- High correlation between predictors (multicollinearity) can make:
 - coefficient estimates unstable,
 - standard errors large,
 - interpretation difficult.

Correlations between polynomials:

```
round(cor(M[, -1]), 3)
```

	I(x ¹)	I(x ²)	I(x ³)
I(x ¹)	1.000	0.984	0.951
I(x ²)	0.984	1.000	0.991
I(x ³)	0.951	0.991	1.000

What is the possible solution?

A possible solution: Orthogonal polynomial Expansion

- Raw polynomial basis functions are transformations of x , such as x , x^2 , and x^3 .
- Raw polynomial basis functions can be highly correlated with each other.
- **Orthogonal** polynomial basis functions are transformed so they are uncorrelated.
 - No overlap in the information they carry
- In R, `poly(x, degree = 3)` creates **orthogonal** polynomial features up to degree 3.
- The fitted curve is the same as using raw polynomial terms, but the coefficients are more stable.

Example

```
(P <- model.matrix( ~ poly(x, 3), data = data.frame(x = 1:4)))
```

```
(Intercept) poly(x, 3)1 poly(x, 3)2 poly(x, 3)3
1           1 -0.6708204          0.5 -0.2236068
2           1 -0.2236068         -0.5  0.6708204
3           1  0.2236068         -0.5 -0.6708204
4           1  0.6708204          0.5  0.2236068
attr(,"assign")
[1] 0 1 1 1
```

Correlations

```
round(cor(P[, -1]), 3)
```

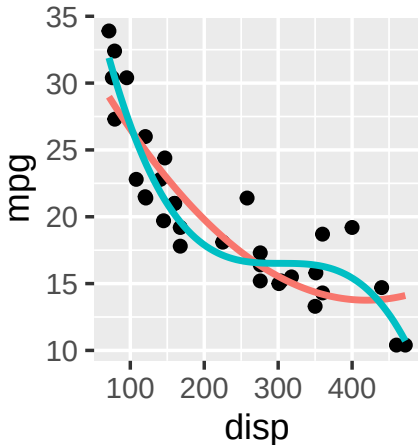
```
           poly(x, 3)1 poly(x, 3)2 poly(x, 3)3
poly(x, 3)1          1          0          0
poly(x, 3)2          0          1          0
poly(x, 3)3          0          0          1
```

Example: Polynomial Regression on mtcars data

- mpg: miles per gallon (fuel efficiency)
- disp: engine displacement (engine size)
- We compare polynomial regression models of different degrees.
- Higher-degree polynomials allow more flexible curves.

Example: Polynomial Regression on mtcars data

```
ggplot(mtcars, aes(displacement, mpg)) + geom_point() +  
  geom_smooth(aes(color = "Degree 2"), method = "lm", formula = y ~ poly(x, 2),  
    se = FALSE) +  
  geom_smooth(aes(color = "Degree 3"), method = "lm", formula = y ~ poly(x, 3),  
    se = FALSE) +  
  labs(color = "Polynomial degree")
```



Polynomial degree



Degree 2

Degree 3

Potential problems with polynomials

- High-degree polynomials are flexible and can overfit.
- The fitted curve may wiggle too much, especially near the edges.
- Polynomial fits are global, so local changes can affect the whole curve..

A possible solution: Splines

What are splines?

- Piecewise polynomial regression methods
- Used for flexible curve fitting
- Different polynomial functions are fitted in different regions of x
- Pieces are joined smoothly at selected points (knots)
- Examples: B-splines, Natural splines, Smoothing splines, and many more.

B-splines vs Natural splines vs Smoothing splines

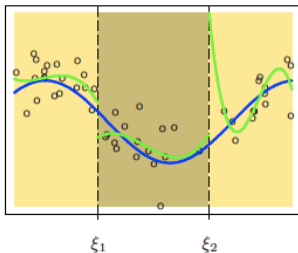
- All are piecewise polynomial models and have local control.
- B-splines
 - Fully flexible within each region
 - No boundary constraints
- Natural splines
 - Add constraints at the boundaries
 - More stable behavior at the extremes
 - Trade a bit of flexibility for better behavior at the edges.
- Smoothing splines
 - Control wiggleness by penalizing curves that bend too much
 - Help the model stay smooth and avoid overfitting.

B-splines

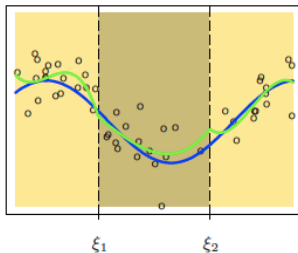
- Choose a set of knots ξ that divide the range of X into non-overlapping regions
- Fit low-degree polynomials (usually cubic) within each region (locally)
- Join the pieces smoothly at the knot $x = \xi$:
 - same value (left polynomial value = right polynomial value)
 - same first derivative (slope from the left = slope from the right)
 - same second derivative (Curvature from the left = curvature from the right)

B-splines

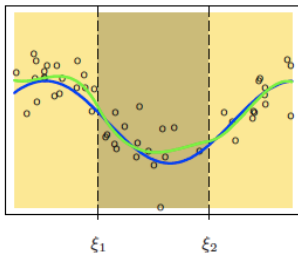
Discontinuous



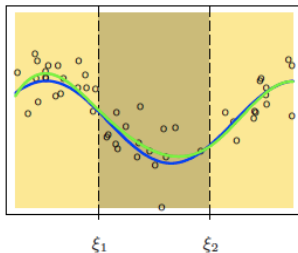
Continuous



Continuous First Derivative



Continuous Second Derivative



Fitting cubic splines in R: `bs()` and `ns()`

Create spline basis functions with the `splines` package

```
bs(x, df = NULL, knots = NULL, degree = 3)  # B-Spline  
ns(x, df = NULL, knots = NULL)              # natural cubic spline
```

- `degree = 3` gives a **cubic spline** (built from cubic polynomial pieces)
- `df` controls flexibility (number of basis functions)
 - larger ``df`` --> more wiggles
 - smaller ``df`` --> smoother curve
- `knots` are the points where spline pieces join
- `ns()` fits a natural cubic spline - linear beyond the boundary knots - often more stable at the extremes

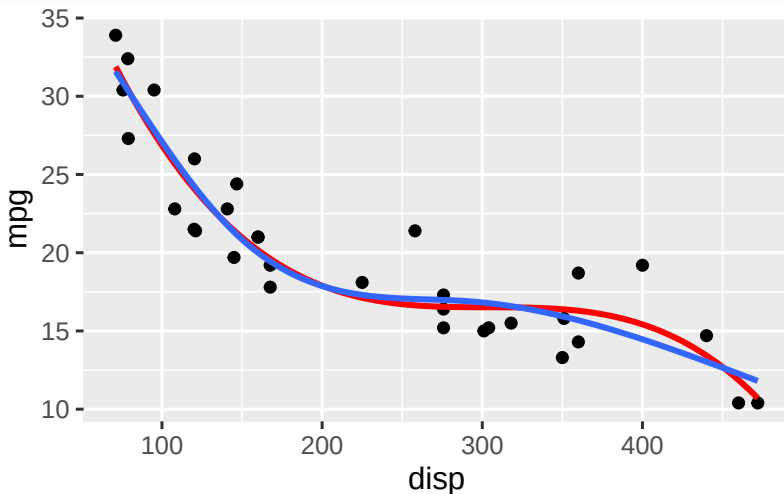
Basis matrix cubic spline with $df = 4$

```
BS <- bs(1:4, df = 4)
# bs() creates a B-spline basis for x.
# If df is given and knots are not,
# R chooses the internal knots automatically,
# typically using quantiles of x.
BS
      1      2      3      4
[1,] 0.00000000 0.0000000 0.00000000 0.00000000
[2,] 0.51851852 0.3703704 0.07407407 0.00000000
[3,] 0.07407407 0.3703704 0.51851852 0.03703704
[4,] 0.00000000 0.0000000 0.00000000 1.00000000
attr("degree")
[1] 3
attr("knots")
[1] 2.5
attr("Boundary.knots")
[1] 1 4
attr("intercept")
[1] FALSE
attr("class")
[1] "bs"      "basis"    "matrix"
```

- Each column is one basis function and each row corresponds to one observation
- With $df = 4$, there are 4 basis functions

Examples

```
ggplot(mtcars, aes(displ, mpg)) + geom_point() +  
  geom_smooth(method="lm", formula= y ~ bs(x, 3), col = "red", se = F) +  
  geom_smooth(method="lm", formula= y ~ ns(x, 3), se = F)
```



Regression trees: A different approach to non-linearity

Regression trees

- Unlike polynomials and splines, trees do not fit smooth curves
- They model the relationship by splitting the feature space into regions
- They can capture abrupt changes and interactions automatically

Binary Recursive Partitioning Algorithm

1. Split the feature space into regions using simple rules, such as $x_j < c$

2. In each region, called a **leaf**, predict the mean response:

$$\hat{y}_R = \frac{1}{n_R} \sum_{i \in R} y_i$$

3. Choose the split that minimizes the **mean squared error (MSE)**.

4. Repeat splitting until a stopping rule is met, such as:

- minimum node size
- maximum tree depth
- no meaningful improvement in MSE

5. **Remark:** Too many splits can lead to overfitting.

6. R packages such as `rpart` perform these steps automatically.

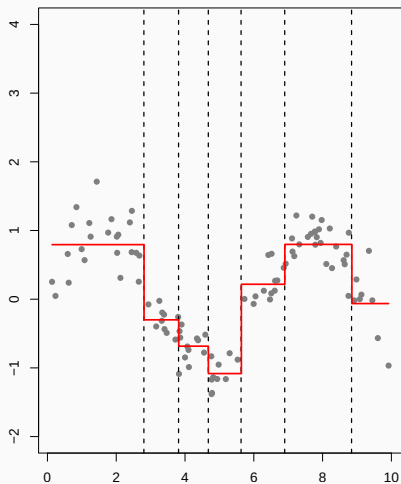
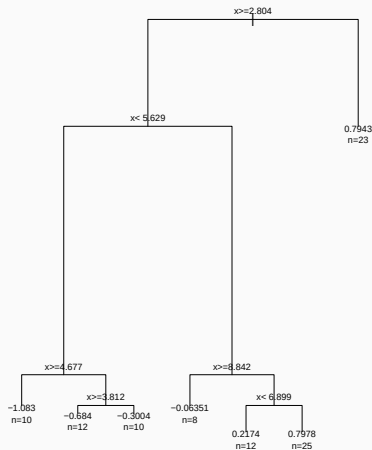
Regression trees in R

Fit a regression tree using `rpart()` from the `rpart` package

```
reg_tree <- rpart(y ~ x, method = "anova")  
# method = "anova" fits a regression tree (continuous response y)  
plot(reg_tree)  
text(reg_tree)
```

- `rpart()` fits a recursive partitioning tree.
- `y ~ x` uses `x` to predict `y`.
- Warning: trees tend to overfit, more on this in classification
- Each terminal node predicts the mean of `y` in that region

Example with one feature: Tree representation (left) and its predictions (right)



Example with two features

Different way of looking at interactions

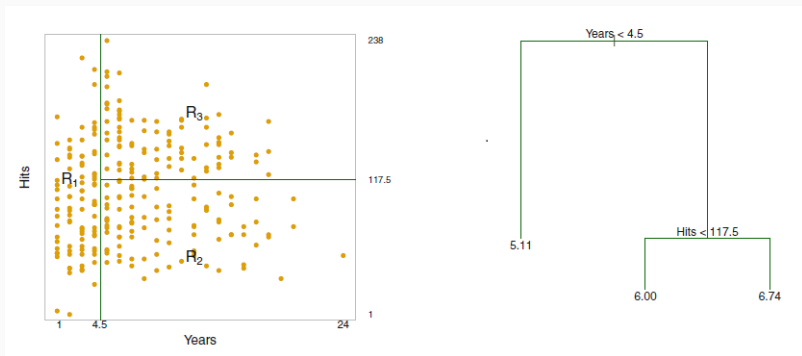
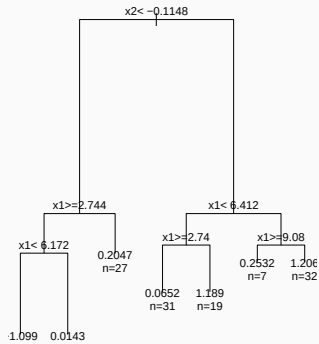
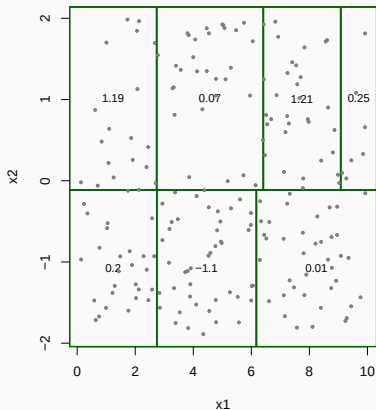


Figure 1: Salaries of baseball players (ISLR)

Example with two features: Tree partitions in the predictor space and the corresponding regression tree

Tree partition in (x1, x2)



Take-home messages

- **Basis expansion** allows linear models to capture complex relationships
- **Polynomials**
 - Add global flexibility
 - Can behave poorly at the boundaries
 - May suffer from multicollinearity
- **Splines**
 - Provide **local flexibility**
 - More stable than high-degree polynomials
 - Natural splines improve boundary behavior
 - Smoothing splines control smoothness instead of requiring fixed knots
- **Regression trees**
 - Model non-linearity by **splitting** the feature space
 - Capture interactions automatically
 - Make no smoothness assumption
 - Produce piecewise-constant predictions

Choose the method based on:

- Smoothness vs interpretability
- Global vs local behavior
- Bias-variance trade-off

Preview Lab 2A

In this lab we'll focus on polynomials and splines. Trees will return with classification.